Proof Pursuit — system architecture

Proof Pursuit — multi-agent system for verified mathematical research

AI-assisted mathematical research competition: 7 hours, 4 problems × 6 cells of increasing difficulty (1/2/3/5/8/13 points). Only verifiable proofs and certificates count. **Nothing is sent to the platform from the repo**: submissions remain drafts in `problema-N/submission/` until a human approves them.

This README explains the entire pipeline: from the documents with the formalized problems, to the loop between the agents, to how and why each agent decides, up to the structured output (submissions + LaTeX report) to be uploaded to the platform.

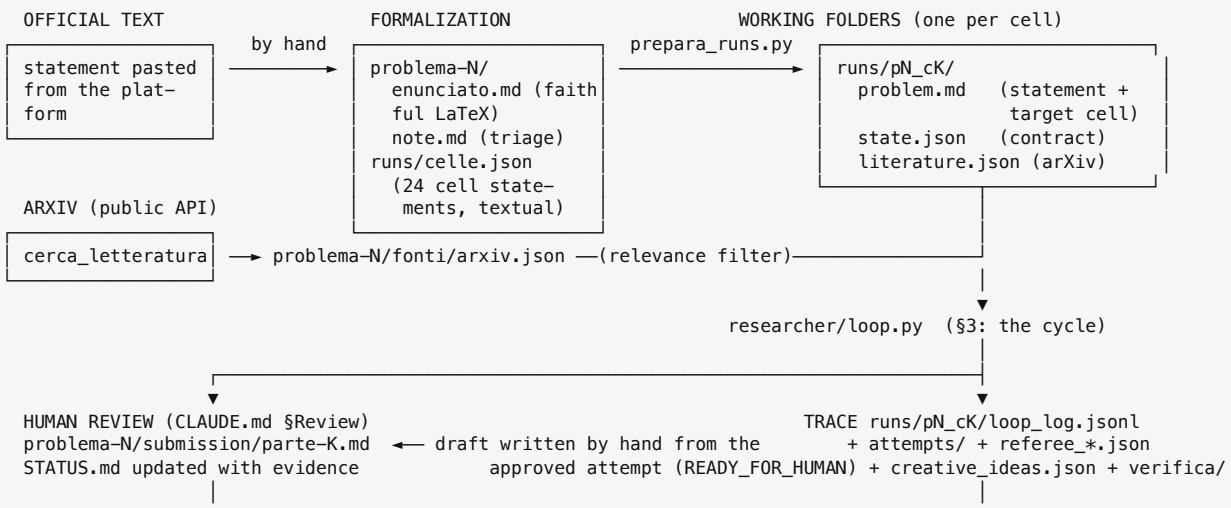
---

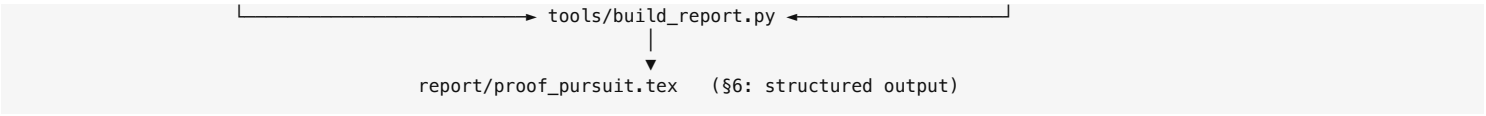
1. Repo map

```
CLAUDE.md            working protocol (applies to humans and agents): what a proof is, what it is not
STATUS.md            status of the 24 cells with evidence; decision log
problema-N/          enunciato.md (official text in LaTeX) · note.md (triage, approaches) · fonti/ (literature read,
                     arxiv.json) · esperimenti/ (research) · certificati/ (verification, separate) · submission/ (drafts)
shared/schemas/      the CONTRACT between the agents: state · attempt · referee_report · creative_ideas (JSON Schema)
researcher/          Researcher (researcher.py), arXiv search (literature.py), bridge to the Referee
                     (bridge_referee.py), end-to-end cycle (loop.py)
referee/             Referee (gabundos): two model judges in parallel + exact checks + deterministic merge
creative/            Creative (Marco Renzi): analyzes the history of attempts and proposes new directions
runs/                one folder per cell (pN_cK): problem.md, state.json, attempts/, referee_report.json,
                     creative_ideas.json, loop_log.jsonl (the trace of decisions), verifica/ (re-runs)
tools/               prepara_runs.py · lancia_loop.sh · cerca_letteratura.sh · build_report.py · run_tests.sh ·
                     autoloop.py (numerical harness) · render.sh
report/              generated LaTeX report (proof_pursuit.tex) with proofs, decisions, code and citations
tests/               tests of the Researcher, the bridge, the loop and Referee B (all without models)
SPIEGAZIONE_SEMPLICE.md   the four problems and the method explained with metaphors
```

---

2. The complete pipeline

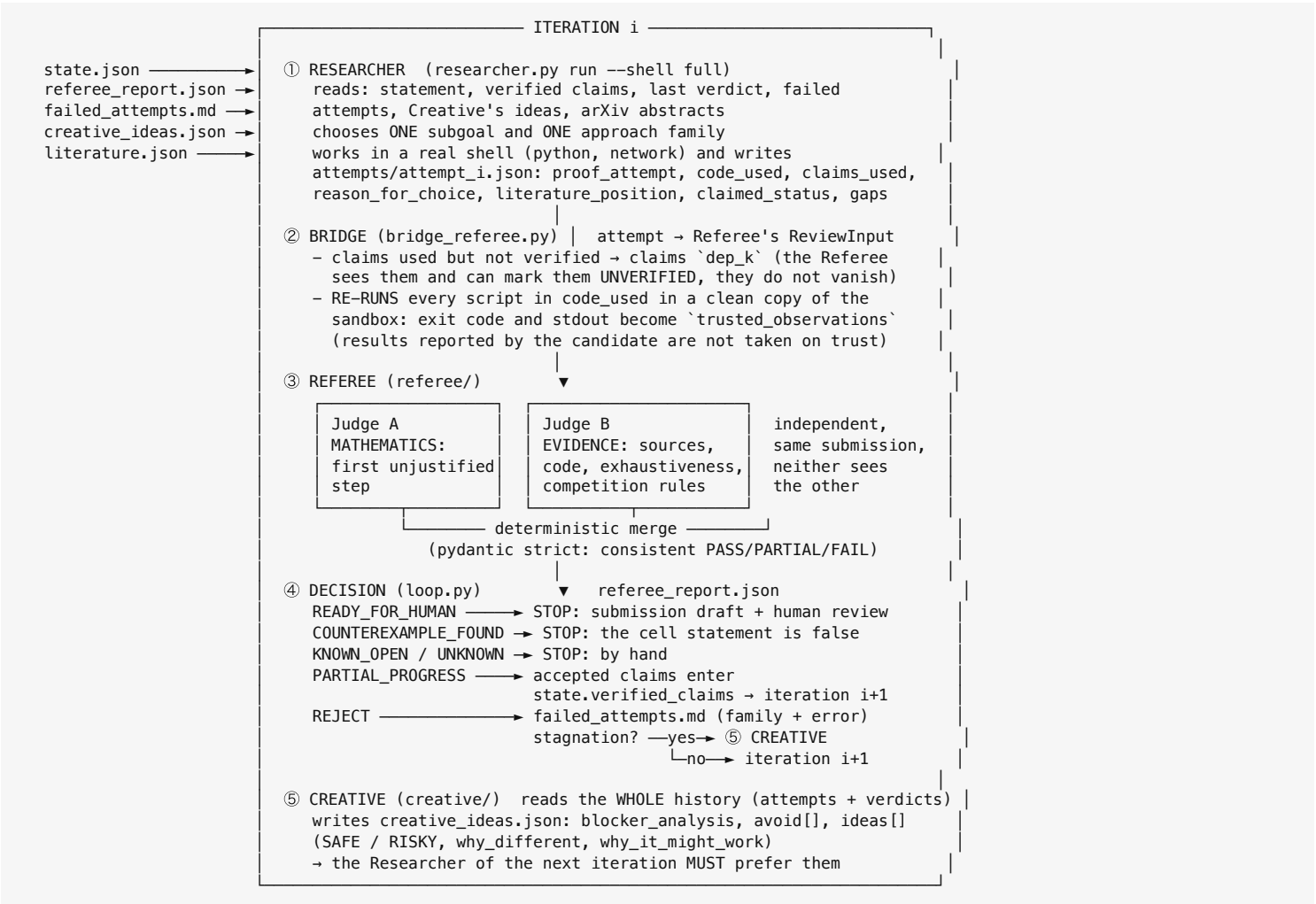




Fixed points of the pipeline: - **The statement is protected.** The cell text enters `state.json` as `cell_statement` and the Referee judges *that* text: no agent can rephrase the target. - **The literature is deterministic.** `literature.py` queries arXiv (exact phrase, real ids, abstracts read) before the attempt; the Researcher must declare in `literature_position` whether it follows, adapts or departs from the state of the art and why. A result taken from a paper is CITED, not proven. - **No agent ever declares "solved".** The Referee reaches at most `READY_FOR_HUMAN`; `ACCEPT` is a human signature.

---

3. The cycle of a cell, iteration by iteration



Each iteration leaves a line in `runs/pN_cK/loop_log.jsonl` with: family and subgoal chosen by the Researcher and **why** (`researcher_reason`, `literature_position`), verdict and **fatal error** of the Referee (`fatal_error`, `next_blocker`), whether and **why** the Creative stepped in (`creative`, `creative_analysis`). It is this trace that ends up in the submission and in the LaTeX report (§6).

---

4. Who decides what, when and why

Moment	Who	Decision	On what basis	Where it is recorded
before the loop	human	formalizes the statement, fixes the cell and the rules	official text, `CLAUDE.md`	`problema-N/enunciato.md`, `runs/celle.json`, `shared/competition_rules.example.json`
start of iteration	Researcher	ONE subgoal, ONE family (`approach_family`)	state, last `fatal_error`, failed attempts, Creative's ideas, arXiv abstracts	`attempt_i.json: subgoal, reason_for_choice, literature_position`
during the attempt	Researcher	which computations to do and with what rigor (`exact` / `interval` / `float_exploration_only`)	competition rules (< 10 min, justified finite set)	`attempt_i.json: code_used[]`, `sandbox/`
end of attempt	Researcher	honest `claimed_status` and `self_reported_gaps`	what it has actually justified	`attempt_i.json`
bridge	orchestrator	re-run the code; record unverified claims as `dep_k`	the Referee judges only trusted observations	`verifica/attempt_i/osservazioni.json`, `review_input.json`
judgment	Referee A	mathematical PASS / PARTIAL / FAIL, **first** fatal error	only the proof, like a journal referee	`attempts/packet_i.json: math_review`
judgment	Referee B	PASS / PARTIAL / FAIL on evidence; provenance of each claim	sources, trusted observations, rules	`attempts/packet_i.json: evidence_review`
merge	Referee (code)	final verdict, `review_status`, accepted claims, `stagnation_signal`	deterministic rules, no model	`referee_report.json`
decision	loop.py	stop / continue / integrate claims / call the Creative	table in §3	`loop_log.jsonl`
stagnation	Creative	analysis of the block, what to avoid, 2–4 new	the whole history of	`creative_ideas.json`, `creative/activation_*.json`

Moment	Who	Decision	On what basis	Where it is recorded
		directions	attempts	
after the STOP	human	write the submission draft, critical review, `STATUS.md`	`CLAUDE.md` protocol	`problema-N/submission/parte-K.md`

Why and when the Creative steps in

The Creative does **not** run at every iteration: it would cost and scatter effort. It steps in only when the cycle is spinning idly, and the loop recognizes this through three independent signals, each sufficient on its own (`loop.py: serve_creative`):

1. **the Researcher asks for it** (`request_creative: true`): it believes it has exhausted the natural avenues;
2. **the Referee signals it** (`stagnation_signal`): the same fatal error keeps repeating;
3. **the stagnation counter rises** (`state.stagnation_count`): the Researcher itself detects, before attempting, that the latest attempts share the same family and the same reason for rejection (lexical similarity of the `fatal_error`).

When it steps in, it reads the *whole* history (not just the last attempt), writes why the block keeps repeating (`blocker_analysis`), what not to repeat (`avoid`) and genuinely different directions with declared risk (`ideas`). The Researcher of the next iteration sees these ideas in the prompt and must say which one it follows and why. The Creative **certifies nothing and never declares a cell solved**: that remains the job of the Referee, then of the human.

Why two judges and a model-free merge

A single model saying “correct” is not a proof. The Referee separates two questions that are readily confused: *is it true?* (A: logic, missing cases, target mismatch, upper bound passed off as exact value) and *is it proven with what has been shown?* (B: sources retrieved or not, computations reproduced or not, exhaustive or sampled, float or exact). Neither judge sees the other. The merge is deterministic code with consistency rules: a PASS with a fatal error is rejected by the validator; a judge that cannot judge returns a `limitation`, which is never a FAIL. If a judge attaches a reservation to a valid report, the bridge moves it into the notes instead of losing the judgment (this was the real case of $U(Q_5)$: a well-motivated PASS discarded for one field too many). Similarly, if B classifies the already-verified claims too, the bridge keeps only the candidate’s ones, as the merge validator requires. The normalizations only remove, they never add. `bridge_referee.py review --replay` re-merges the last saved raw reports without calling the models again: useful after a fix to the bridge.

---

5. The contracts (shared/schemas)

- `state.schema.json` — `current_target`, `cell_statement` (protected), `verified_claims[]`, `failed_attempts[]`, `stagnation_count`, `cell_status` with allowed states SOLVED · SOLVED_BY_COMPUTATION · PARTIAL_PROGRESS · UNRESOLVED · KNOWN_OPEN · FALSE. The Researcher touches only `stagnation_count`; `highest_verified_cell` is moved only by the Referee.
- `attempt.schema.json` — all fields mandatory (`literature_position` included); `claimed_status` ∈ CELL_SOLVED_CANDIDATE · LEMMA_CANDIDATE · NUMERICAL_EVIDENCE_ONLY · LITERATURE_ONLY · COUNTEREXAMPLE_CANDIDATE · NO_PROGRESS.
- `referee_report.schema.json` — `verdict`, `review_status` (READY_FOR_HUMAN ≠ ACCEPT), `fatal_error`, `next_blocker`, `accepted_claims`, `stagnation_signal`.
- `creative_ideas.schema.json` — `blocker_analysis`, `avoid[]`, `ideas[]` with risk and motivations.

---

6. Structured output for the platform

For each cell the platform receives a written submission. Here the path is:

1. **Loop STOP with READY_FOR_HUMAN** ⇒ a human writes `problema-N/submission/parte-K.md` following `CLAUDE.md`: result and scope · complete proof (or certificate + justification) · verification instructions with measured times · sources (distinguishing existing results and our contribution) · limitations.
2. **Human critical review** (missing hypotheses, circularity, code/statement discrepancies) ⇒ `STATUS.md` moves to `revisionato`. A READY_FOR_HUMAN verdict without review remains `bozza pronta`.
3. **LaTeX report** `tools/build_report.py` ⇒ `report/proof_pursuit.tex`, for each problem and cell: official statement · status · submission (proof) · **agents' decisions** iteration by iteration (family, reason, verdict, fatal error, Creative's intervention and its analysis) · **code** the proof rests on, with the outcome of the trusted re-run and the Referee's verdict · **arXiv citations** found by the deterministic search and used by the Researcher, with the declared position relative to the state of the art.

Golden rule (CLAUDE.md): PROVEN / CITED (with source) / VERIFIED ON A FINITE RANGE / CONJECTURE are four different things and the submission keeps them separate.

---

7. How to use it

```
python3 -m venv .venv && .venv/bin/pip install numpy mpmath pydantic     # ambiente
tools/run_tests.sh                                                        # tutte le suite, senza modelli
tools/cerca_letteratura.sh                                                # arXiv → problema-N/fonti/arxiv.json
.venv/bin/python tools/prepara_runs.py p1 p2                              # cartelle runs/pN_cK
tools/lancia_loop.sh p1 2               # loop reale su tutte le celle di P1, 2 iterazioni, in parallelo
tools/lancia_loop.sh p2 2 3 4 5 6       # P2, solo celle 3–6 (1–2 già chiuse)
.venv/bin/python researcher/loop.py --workdir runs/p1_c1 --max-iter 3 --researcher-backend cli --shell full \
      --referee cli --creative cli      # una cella sola, a mano
.venv/bin/python tools/build_report.py   # → report/proof_pursuit.tex (+ HTML di anteprima se c'è pandoc)
```

Operational notes: the Researcher with shell `full` runs with pre-authorized permissions and must be started from a human terminal (in Claude Code: `!` prefix). Backend: `claude -p --json-schema` with a subscription; no API key needed. Testing the cycle without models: `--researcher-backend mock --referee mock --creative rule_based`.

8. Minimum rules for collaborating

1. Branch per person/part, small PRs to `main`; `STATUS.md` is updated in the same PR.
2. Clean code: small functions, comments in Italian saying what and why, no duplicated dependencies.
3. Every experiment: reproducible command, seed, times. Every certificate: separate from the research.
4. No submission to the platform from the repo.